

Decision Transformer for Sequential Portfolio Allocation: An Offline Reinforcement Learning Study on 30 Years of Market Data

Md. Shahriar Islam Bhuiyan
Section: <section> Student ID: <id>
shojeb.bhuiyan@gmail.com

Abstract

Allocating capital across many assets is a sequential decision problem, but the obvious way to solve it with reinforcement learning—letting an agent explore a live market—is impossible: every exploratory trade costs real money and no historical regime can be replayed. Offline reinforcement learning and, more recently, return-conditioned sequence modelling promise a way out, and the Decision Transformer (DT) in particular claims that simply conditioning an autoregressive model on a desired Return-to-Go (RTG) is enough to recover good behaviour from a fixed dataset. In this project we test that claim on a long-only, cost-aware, multi-asset allocation task built from a 30-year daily-close dataset. We construct a 14-asset investable universe with 6,369 trading days, engineer a 2,580-dimensional state from 129 daily features over a 20-day lookback, synthesise 3,600 offline trajectories from a mixture of 18 behaviour policies, and train eight learned allocators—Decision Transformer, transformer behaviour cloning, TD3+BC, IQL, CQL, PPO, SAC and A2C—across five random seeds each, for a total of 40 checkpoints per universe, then backtest all of them against six classical baselines on a strictly held-out 2020–2025 window with a 10 bp proportional transaction cost. The results are consistently negative for the learned allocators and informative about why: no learned method beats the cost-aware equal-weight portfolio (Sharpe 1.118); DT reaches Sharpe 0.966 ± 0.133 and is statistically indistinguishable from its own RTG-free control (0.976 ± 0.106); sweeping the target RTG across training-return quantiles moves realised Sharpe by less than 0.04 and in the wrong direction, showing that the return-conditioning channel is essentially unused; a weight-level analysis explains the shortfall—on four of five seeds DT’s average book is equal weight plus noise (Herfindahl 0.0715 against a uniform 0.0714), so the model rediscovers the baseline’s allocation and then pays turnover to keep re-deriving it; a transaction-cost ablation shows DT still trails equal weight by 0.07 Sharpe even at zero cost, so turnover explains only about half of the gap; and a context-length ablation over $K \in \{10, 20, 30, 50\}$ moves the imitation loss by 0.3% while closing none of the gap. Repeating the entire pipeline on a second universe that adds Bitcoin and aluminium reproduces every qualitative conclusion.

1 Introduction

Portfolio allocation asks how to split one unit of capital across N assets on every trading day so that long-run wealth grows quickly without exposing the investor to ruinous drawdowns. It is naturally sequential: today’s weights determine tomorrow’s starting holdings, and every change of mind is taxed by transaction costs, so a decision cannot be evaluated in isolation. This makes the problem a textbook fit for reinforcement learning (RL), and a poor fit for the standard RL *protocol*. An online agent learns by acting, and acting in a market means losing real money on

every exploratory trade; worse, markets are non-stationary and non-repeatable, so the environment an agent explored last year no longer exists. The practical consequence is that essentially all realistic financial RL is *offline*: the agent must learn from a fixed record of what some other strategy did. Getting this right matters well beyond finance—pension funds, index providers and robo-advisors already allocate trillions using rules that a learned allocator would have to beat—and it is also a demanding stress test for offline RL itself, because the environment is exogenous (the agent cannot move prices), the reward is nearly a martingale, and a trivial policy (hold $1/N$ forever) is a famously strong baseline [1].

A long line of work has attacked this problem. Moody and Saffell [2] introduced direct (policy-gradient) reinforcement for trading and showed that optimising a differentiable Sharpe-like utility beats training a return forecaster and trading its output; their setting, however, is single-asset and predates deep function approximation. Deng et al. [3] added deep representation learning on top of direct RL and improved signal extraction, but still on individual futures contracts and without an explicit budget constraint. Jiang et al. [4] are the closest classical antecedent of our setup: they define the portfolio-vector-memory MDP with simplex actions and a transaction-cost-adjusted log return, which we adopt almost unchanged, but they evaluate on a short cryptocurrency window in which nearly every strategy was profitable, so the baseline bar was low. Zhang et al. [5] ran DQN, PG and A2C over 50 liquid futures with careful cost modelling and reported genuine but small risk-adjusted gains—their study is methodologically the most careful of the group, and notably also finds that performance degrades quickly as costs rise. Yang et al. [6] ensembled PPO, A2C and DDPG for 30 Dow stocks and report a Sharpe above the index, but their agents interact with a simulator built from the same prices they are evaluated on, which is exactly the online protocol that is unavailable in practice; the accompanying FinRL library [7] has made this protocol the de facto standard and, in our view, has propagated its optimism. On the algorithmic side, offline RL has developed principled answers to the distribution-shift problem that dooms naive off-policy learning on static data [8]: CQL [9] penalises over-estimated Q -values on out-of-distribution actions, IQL [10] avoids querying unseen actions altogether by fitting an expectile value function, and TD3+BC [11] shows that a single behaviour-cloning term added to TD3 [12] is a surprisingly strong baseline. Orthogonally, the Decision Transformer [13] and Trajectory Transformer [14] discard value learning entirely and recast offline RL as autoregressive sequence modelling over $(\hat{R}_t, s_t, a_t)$ tokens with a GPT-style causal transformer [15]; conditioning on a high target return at inference is supposed to elicit high-return behaviour. Their published evidence is drawn from Atari, D4RL locomotion and Key-to-Door—dense, stationary, endogenous environments that differ from markets in every property that matters here. Closest to our architecture, Kisiel and Gorse [16] apply an encoder-decoder transformer directly to asset allocation and train it end to end on a transaction-cost-penalised Sharpe objective, outperforming LSTM alternatives; theirs, however, is a supervised allocator rather than a return-conditioned sequence model, so it tells us nothing about whether the Return-to-Go channel carries information. Studies that do apply Decision Transformers to allocation typically report favourable numbers against a single index benchmark, on one asset universe, with one seed, and without an RTG-free control, so it is impossible to tell whether the transformer or the return conditioning did any work.

That gap is what this project addresses. Every ingredient needed to answer the question honestly—an RTG-free control that is architecturally identical to DT, several seeds, a transaction cost, a genuinely held-out multi-year window, classical baselines that are actually competitive, and significance testing that accounts for the number of strategies tried—is individually standard, yet we are not aware of a portfolio study that combines them. Without the RTG-free control, DT’s score cannot be separated from plain imitation of the behaviour data; without several seeds, a difference of 0.1 Sharpe is noise; without a transaction cost, any high-turnover policy looks good;

and without a strong $1/N$ baseline, a mediocre allocator looks like a success. We therefore rebuilt the pipeline end to end so that each of these controls is present simultaneously, and we report what it shows even though the headline finding is negative. A negative result obtained under proper controls is more useful to the next practitioner than a positive one obtained without them, and the ablations tell us specifically *which* link in the DT story breaks on this task.

1.1 Our Contribution

1. We formalise long-only, cost-aware, daily multi-asset allocation as a finite-horizon MDP with simplex actions and a net-log-growth reward, and implement it as a dependency-free NumPy environment in which every strategy—classical, cloned or learned—is the same callable $(t, s_t) \mapsto a_t \in \Delta^{N-1}$, so all comparisons run through identical backtest code.
2. We build a reproducible offline dataset from a public 30-year daily-close dataset [17]: NYSE-calendar alignment, a synthetic 10-year Treasury total-return series derived from yields, a floored T-bill cash asset, 129 engineered daily features flattened over a 20-day lookback into $s_t \in \mathbb{R}^{2580}$, and 3,600 episodes of 252 days rolled out from a mixture of 18 behaviour policies.
3. We train and evaluate **eight** learned allocators (Decision Transformer, transformer BC, TD3+BC, IQL, CQL, PPO, SAC, A2C) at **five seeds** each and compare them with **six** classical baselines under an identical protocol, giving 40 learned checkpoints and 46 evaluated strategies per universe.
4. We report a full ablation suite: RTG conditioning on/off (DT vs. an architecturally identical control), target-RTG quantile, Decision Transformer context length $K \in \{10, 20, 30, 50\}$, transaction cost $\mu \in \{0, 5, 10, 25\}$ bp, and an entire second asset universe that adds Bitcoin and aluminium.
5. We quantify significance properly—circular block-bootstrap confidence intervals on Sharpe differences, the Ledoit–Wolf robust Sharpe test [18], and the deflated Sharpe ratio [19]—and document the RL correctness bugs (self-referential Bellman targets, frozen target networks, a batch-wise instead of action-wise CQL penalty) that had to be fixed before any of the baseline numbers were meaningful.

2 Material and Methods

Figure 1 is the graphical abstract of the project. The pipeline runs strictly left to right and every stage hands off through files on disk rather than in-memory objects, so any stage can be re-run on its own once the previous stage’s artefacts exist. **(1) Data preparation** reads the three raw CSVs, reindexes the union calendar onto NYSE trading days (defined as days on which the S&P 500 close is present), forward-fills gaps for at most five sessions, synthesises a 10-year Treasury total-return index from the yield series and a cash index from the 13-week T-bill yield, and splits the columns into a 14-asset *investable* universe and a larger *feature-only* set. **(2) Feature engineering** turns those prices into 129 daily features—returns, MACD, RSI, Bollinger %b, realised volatility at two horizons, cross-sectional momentum ranks, macro indicators and crisis-event flags—then flattens a 20-day sliding window into a single state vector $s_t \in \mathbb{R}^{2580}$ and standardises it with training-split statistics only. **(3) Trajectory synthesis** rolls 18 behaviour policies through the portfolio environment over 200 sampled 252-day windows each, producing 3,600 episodes of $(\hat{R}_t, s_t, a_t, r_t)$ tuples. **(4) Offline training** fits the Decision Transformer and seven baselines to that buffer,

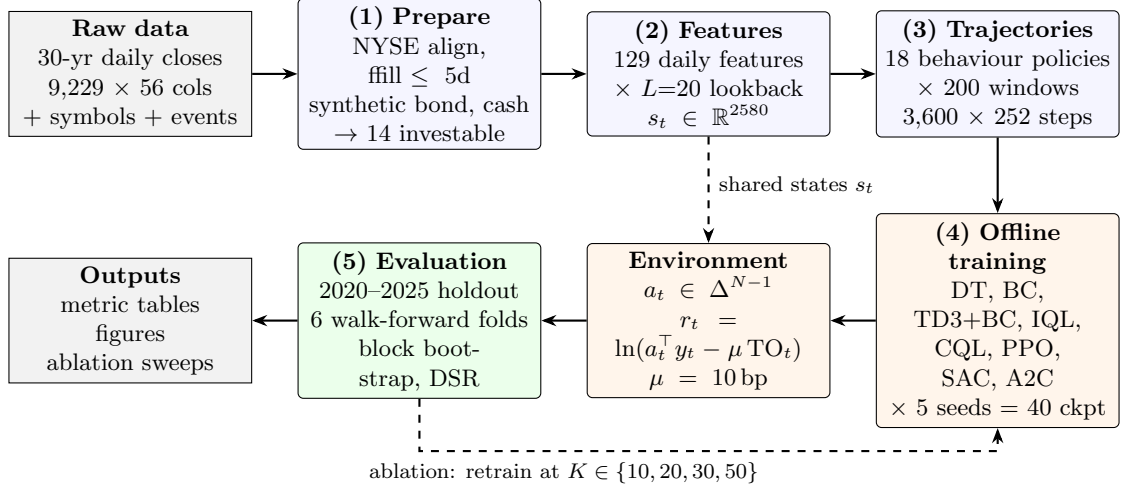


Figure 1: Graphical abstract of the pipeline. Stages (1)–(3) turn a raw 30-year daily-close dataset into a 3,600-episode offline buffer of $(\hat{R}, s, a, r)$ tuples; stage (4) fits eight learned allocators at five seeds each; stage (5) replays every checkpoint through the *same* cost-aware portfolio environment used to generate the data, on a window the learners never saw. Every arrow is a file on disk, so each stage is independently reproducible. The dashed loop is the context-length ablation, which is the only part of the study that requires retraining.

five seeds apiece, writing one checkpoint per (model, seed) pair plus a manifest. **(5) Evaluation** loads every successful checkpoint, wraps it as a policy callable, and replays it through the same environment on the held-out window, producing metric tables, significance tests, walk-forward folds and the ablation sweeps. The dashed feedback arrow marks the only loop in the diagram: the ablation stage re-enters training when it needs a model retrained at a different context length.

2.1 Dataset Description

All market data comes from the public Kaggle dataset *30 Years of Stock Market Data* by Asim Islam [17], available at <https://www.kaggle.com/datasets/asimislam/30-yrs-stock-market-data>. It contains three files, summarised in Table 1: a wide matrix of daily closing prices for 56 instruments, a symbol metadata table giving each column’s ticker, asset class and unit, and a table of 11 dated financial crises which we convert into event-window indicator features. We chose this dataset for three reasons. First, its span (31 calendar years) is long enough to contain several distinct market regimes—the dot-com collapse, the 2008 credit crisis, the COVID-19 crash, and the 2022 rate-hike drawdown—so a chronological train/test split is a real distribution shift rather than a formality. Second, it is genuinely multi-asset: equities, mutual funds, ETFs, precious and industrial metals, energy, currencies, crypto and the full Treasury yield curve appear in one aligned frame, which is what makes a *diversified* allocation problem possible at all. Third, it is a single self-contained download, so the entire study can be reproduced without a paid data vendor.

Universe construction. Not every column in the file can be held in a portfolio. Indices such as the DAX or the VIX have no investable total return in this dataset, Treasury columns are *yields* rather than prices, and WTI crude contains a negative close on 2020-04-20 that makes a gross return undefined. We therefore split the columns in two. The **investable universe** (“Universe A”, $N = 14$) consists of nine equity-like instruments (S&P 500 ETF, Fidelity Blue Chip Growth Fund, Apple, Microsoft, Amazon, Nvidia, JP Morgan Chase, Walmart, Fidelity Select Energy Portfolio), three metals (gold, silver, copper), one synthetic 10-year Treasury total-return index built from the

Table 1: Properties of the raw dataset [17] and of the derived Universe A used for all headline results. “Feature-only” columns are observable to the agent through the state vector but are not investable—either because they are indices or yields with no tradable total return in the file, or because they contain invalid prices (WTI crude printed -37.63 on 2020-04-20).

	Property	Value
Raw	Source	Kaggle, <code>asimislam/30-yrs-stock-market-data</code>
	Files	prices, symbol metadata, financial events
	Rows $\times$ columns	$9,229 \times 56$ (daily closes)
	Calendar span	1995-01-02 – 2025-12-30
	Asset classes	indices, stocks, ETFs/funds, metals, energy, FX, crypto, Treasuries
	Financial events	11 dated crises (1997 Asia $\rightarrow$ 2025 tariffs)
Universe A	Investable assets N	14
	Trading days	6,369 (2000-09-01 – 2025-12-30, NYSE calendar)
	State dates after lookback	6,330
	Daily features	129
	Lookback L	20 days $\Rightarrow d_s = 129 \times 20 = 2,580$
	Feature-only series	VIX, USD, WTI, Brent, natural gas, DAX/FTSE/HSI/Nasdaq/NYSE, yields
	Missing-data rule	forward-fill ≤ 5 sessions, then drop the leading window
Split	Train	2000-09-01 – 2016-12-31
	Validation	2017-01-01 – 2019-12-31
	Test (held out)	2020-01-01 – 2025-12-30 (1,507 trading days)

yield series via the duration approximation $r_t^{\text{bond}} \approx -D \Delta y_t^{10Y} + y_{t-1}^{10Y}/252$ with $D = 8$, and one cash asset accruing $\max(y_t^{\text{bill}}/100/252, 0)$ per day (the floor is needed because the 13-week bill yield printed -0.105% on 2020-03-26). Universe A starts on 2000-09-01, the first date on which all three metals are available. Everything else in the file—VIX, the US dollar index, WTI and Brent, natural gas, the four foreign equity indices, and the rest of the yield curve—enters the **feature-only** set: the agent observes it but cannot allocate to it. A second universe (“Universe B”) adds Bitcoin and aluminium and therefore starts on 2014-09-17, the first date on which Bitcoin is quoted; it is used in Section 3.7.5 as a robustness check and is otherwise held out of the headline numbers.

Feature description. The 129 daily features decompose as follows. *(i) Returns* (14 features): the one-day simple return of each investable asset. *(ii) Technical indicators* ($14 \times 5 = 70$): for each asset, the MACD histogram normalised by price (12/26/9 EMA spans), RSI(14), Bollinger %b over a 20-day window, and annualised realised volatility at 20 and 60 days. These are all close-based, which is a constraint of the dataset—it carries no volume, so volume- or microstructure-based indicators are impossible. *(iii) Cross-sectional momentum ranks* ($14 \times 2 = 28$): the percentile rank of each asset’s trailing cumulative return within the universe at 60- and 120-day horizons, which gives the model a relative rather than absolute momentum signal. *(iv) Macro features* (6): the VIX level, the daily US dollar return, the 10Y–3M and 30Y–5Y term spreads, and WTI and natural-gas returns. *(v) Event flags* (11): one binary indicator per crisis in the events file, switched on for a ± 30 -day window around the event date, which lets the model recognise regime boundaries it would otherwise have to infer. A state is the concatenation of these 129 features over the trailing 20 sessions, so s_t contains both the current cross-section and its recent trajectory; all features are computed from information available at or before day t , and a dedicated unit test asserts that shifting any feature forward in time changes the result (a no-lookahead check).

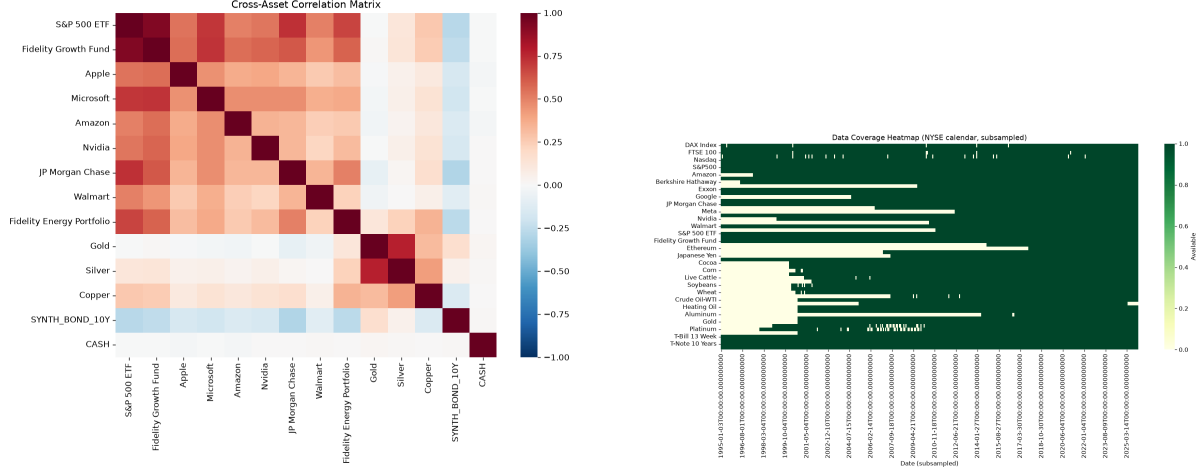


Figure 2: **Left:** daily-return correlation matrix of the 14 investable assets over 2000–2025. The block structure is what makes diversification feasible and is precisely the structure a mean-variance optimiser exploits: the nine equity-like instruments form a tightly correlated cluster ($\rho \approx 0.4\text{--}0.9$), the three metals form a second, weaker cluster, the synthetic 10-year bond is mildly *negatively* correlated with equities, and cash is uncorrelated with everything. **Right:** data-coverage heatmap over calendar time. Coverage is the reason Universe A starts in September 2000 rather than 1995—the metals series begin on 2000-08-30—and the reason Bitcoin and aluminium are quarantined into Universe B.

2.2 Tools / Models / Algorithms Used

The whole project is implemented from scratch in Python 3.12 with PyTorch 2.5 (CUDA 12.1), NumPy, pandas, scikit-learn (for the Ledoit–Wolf covariance estimator) and matplotlib. We deliberately did *not* use FinRL, d3rlpy or stable-baselines3: the correctness issues we discuss in Section 3.8 are exactly the kind that a black-box library hides, and the point of the study is to control the comparison. Training ran on a single NVIDIA GTX 1070 (8 GB, Pascal).

2.2.1 The allocation MDP

We model daily rebalancing as a finite-horizon MDP $\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, P, R, \gamma, \rho_0, T_{\text{ep}} \rangle$ driven by an *exogenous* price process: the agent chooses weights and pays turnover costs, but cannot move prices. Let $y_t = (p_{1,t}/p_{1,t-1}, \dots, p_{N,t}/p_{N,t-1})^\top$ be the vector of gross returns. The action is a long-only, fully-invested allocation on the simplex,

$$a_t = w_t \in \Delta^{N-1} = \left\{ w \in \mathbb{R}^N : w_i \geq 0, \sum_{i=1}^N w_i = 1 \right\}, \quad (1)$$

which forbids shorting and leverage; every model enforces this either with a softmax output head or with a Euclidean projection onto Δ^{N-1} . The state $s_t \in \mathbb{R}^{2580}$ is the standardised flattened feature window of Section 2.1. The reward is the net logarithmic growth of the portfolio after a proportional cost μ on L_1 turnover,

$$R_t = \ln \left(\max \left\{ a_t^\top y_t - \mu \sum_{i=1}^N |a_{i,t} - a_{i,t-1}|, \varepsilon \right\} \right), \quad \mu = 0.001, \varepsilon = 10^{-8}, \quad (2)$$

so that maximising $\sum_t R_t$ maximises terminal wealth; the floor keeps $\ln$ defined on days when costs would exceed gross growth. Return-to-Go is the undiscounted remaining reward,

$$\hat{R}_t = \sum_{t'=t}^{T_{\text{ep}}-1} R_{t'}, \quad \gamma = 1, \quad (3)$$

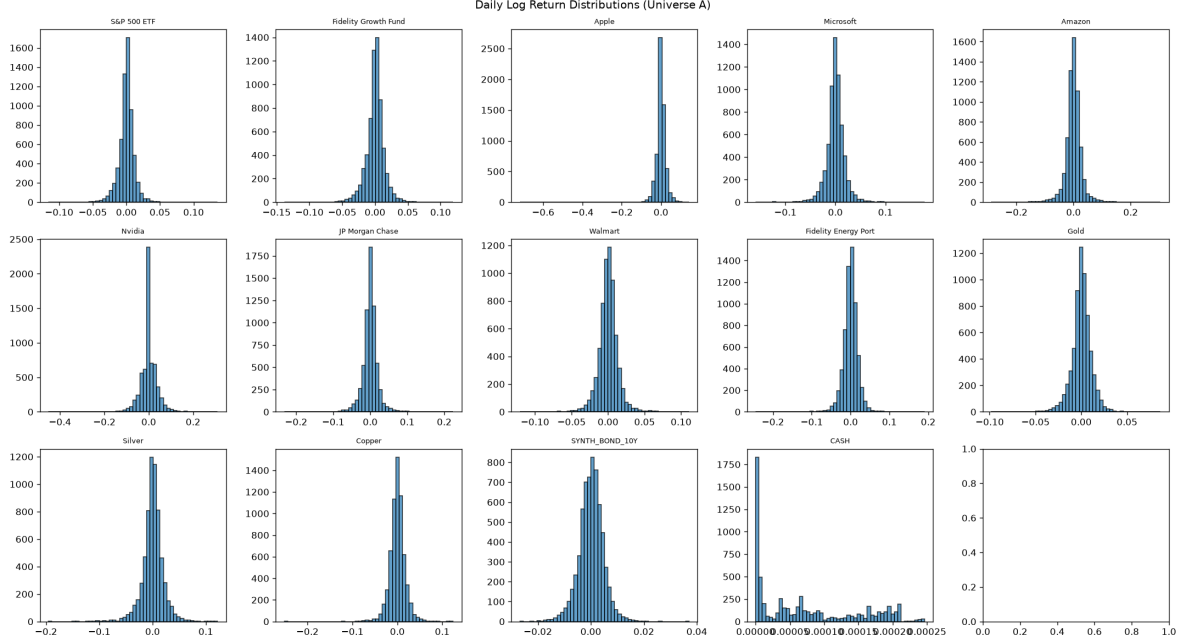


Figure 3: Daily log-return histograms for all 14 investable assets, 2000–2025. Every equity and metal series is leptokurtic with visible fat tails and occasional single-day moves beyond $\pm 20\%$ (Apple, Nvidia, Amazon, the energy fund); the synthetic 10-year bond is much tighter; and cash is a strictly non-negative, near-discrete series pinned to the T-bill yield and floored at zero after 2020. Two consequences drive design choices elsewhere in the paper: the fat tails mean a Gaussian Sharpe test is inadequate and motivate the block-bootstrap and Ledoit–Wolf procedures of Section 2.3, and the crash days make the raw $\ln(\cdot)$ reward divergent, which is why the environment floors the net growth factor at $\varepsilon = 10^{-8}$.

which is exactly the quantity the Decision Transformer conditions on. Training episodes have $T_{\text{ep}} = 252$ days (one trading year).

Behaviour policies and the offline buffer. Because there is no logged trading record to learn from, the offline dataset must be synthesised. We roll out a mixture of 18 policies: six classical allocators (buy-and-hold, equal weight, max-Sharpe mean-variance with a Ledoit–Wolf covariance estimate [20], minimum-variance, inverse-volatility risk parity, and 60-day cross-sectional momentum), nine Dirichlet-perturbed copies of mean-variance/momentum/equal-weight at concentrations $\alpha \in \{5, 15, 50\}$, and three pure random Dirichlet policies at $\alpha \in \{0.5, 1, 2\}$. The deliberate inclusion of bad and noisy policies is what makes the dataset suitable for offline RL at all: a buffer containing only good behaviour cannot teach a value-based method to distinguish good from bad, and it removes any opportunity for DT to “stitch” high-return segments. Sampling 200 windows of 252 days from the training range for each policy gives 3,600 episodes and roughly 8.2×10^5 usable transitions. Trajectories are stored in a compact format in which all episodes index into a single shared state matrix, which reduces the buffer from ~ 9.4 GB to ~ 65 MB and lets the whole dataset live in GPU memory during training.

2.2.2 Model 1: Decision Transformer

The Decision Transformer [13] is our primary model (Figure 4). It interleaves three token types per timestep—the standardised Return-to-Go scalar, the state, and the action—each passed through

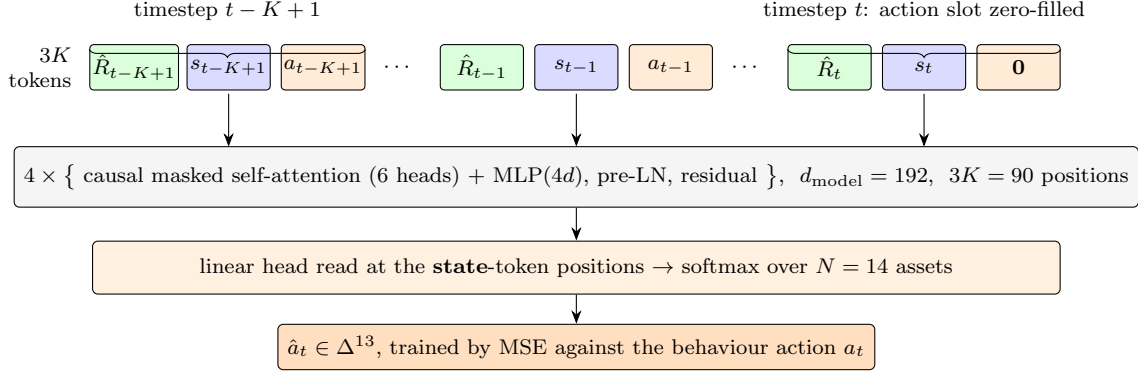


Figure 4: Decision Transformer architecture as implemented. Each timestep contributes three tokens—Return-to-Go, state and action—so a context of $K = 30$ timesteps becomes a 90-token sequence. A causal mask prevents any position from attending to the future, the action head reads only the *state*-token positions (so the prediction at step t sees $\hat{R}_t$ and s_t but not a_t), and a softmax output enforces the simplex constraint exactly. At inference the current-step action slot is zero-filled, since the causal mask already hides it. Transformer BC is byte-for-byte this network with the Return-to-Go channel zeroed, which makes it an exact control for the value of return conditioning.

its own linear encoder into a $d_{\text{model}} = 192$ embedding, added to a learned positional embedding over the $3K$ token slots, and processed by a 4-layer pre-LayerNorm causal transformer with 6 attention heads. Predictions are read off the *state*-token positions and passed through a softmax, which enforces $a_t \in \Delta^{N-1}$ by construction rather than by projection. Training minimises the mean squared error between the predicted allocation at the final context position and the behaviour action recorded in the buffer. At evaluation time the RTG budget is initialised to a chosen quantile of the training episodes’ starting RTG, standardised with the $(\mu_{\text{RTG}}, \sigma_{\text{RTG}}) = (-0.0023, 0.1394)$ statistics computed on the training set, decremented by each realised log reward, and reset every 252 steps so that a six-year rollout stays in the range of remaining-return values the model actually saw during training. We chose DT because it is the method under test: it is the only model here that can, in principle, be *asked* for a return level at inference time, and the entire experimental design is built around checking whether that request has any effect.

2.2.3 Model 2: Transformer Behaviour Cloning (the RTG control)

Transformer BC is the same network with the Return-to-Go channel zeroed, so it has the same 2,298,638 parameters, the same optimiser, the same batches and the same seeds as DT, and differs *only* in whether it can see the return target. This is the single most important baseline in the paper. Any gap between DT and BC is attributable to return conditioning and to nothing else; conversely, if $\text{DT} \approx \text{BC}$ then the transformer is merely imitating the behaviour mixture and the “decision” half of Decision Transformer is inert. We are not aware of a portfolio-allocation study that reports this control, which is one reason we consider the comparison worth publishing even though its outcome is negative.

2.2.4 Models 3–5: offline RL (TD3+BC, IQL, CQL)

These three represent the value-based branch of offline RL and are included because they are the methods DT is supposed to replace (Figure 5). **TD3+BC** [11] is TD3 [12] plus a behaviour-cloning regulariser with weight $\alpha = 2.5$; we chose it because it is the strongest “one-line-change” offline

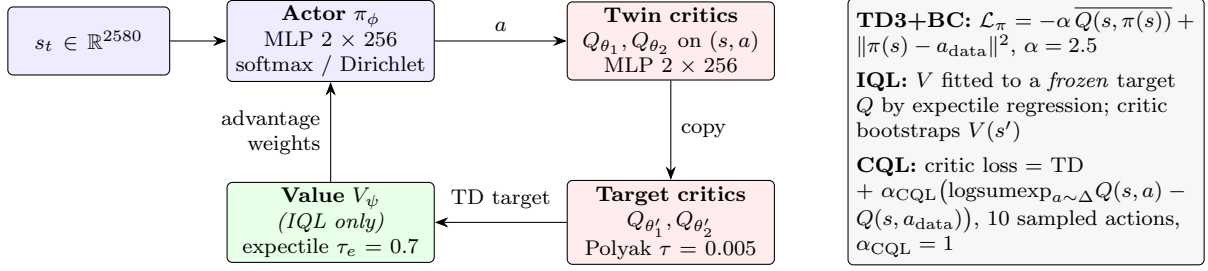


Figure 5: Shared actor-critic skeleton of the three offline RL baselines and the point at which each algorithm departs from it. All three use two-hidden-layer MLPs of width 256, twin critics with Polyak-averaged targets ($\tau = 0.005$), $\gamma = 0.99$, Adam at 3×10^{-4} , and a simplex projection on the actor output. The shared bootstrap target is $r + \gamma(1 - d) \min_i Q_{\theta'_i}(s', \pi(s'))$, with the done flag d cutting the bootstrap at an episode boundary. TD3+BC [11] anchors the policy to the data with an explicit BC term; IQL [10] never queries out-of-distribution actions at all, fitting an expectile value function instead; CQL [9] pushes down Q -values on actions sampled from the simplex relative to the dataset action.

baseline in the literature and therefore the fairest cheap comparison. **IQL** [10] fits an expectile ($\tau_e = 0.7$) value function to a frozen target critic and trains the actor by advantage-weighted regression; we chose it because it never evaluates Q at actions outside the data, which is the most conservative possible response to the distribution shift our noisy behaviour mixture creates. **CQL** [9] adds a conservative penalty with weight $\alpha_{\text{CQL}} = 1$ computed as a log-sum-exp over ten actions sampled from the simplex per state, i.e. the CQL(H) variant; we chose it as the pessimism-based counterpoint to IQL’s avoidance-based approach. All three use $\gamma = 0.99$, Polyak targets with $\tau = 0.005$, Adam at 3×10^{-4} , and two-hidden-layer MLPs of width 256 for both actor and critics.

2.2.5 Models 6–8: online RL reference points (PPO, SAC, A2C)

PPO [21], SAC [22] and A2C [23] are included because they are what the applied literature actually uses for portfolio RL [6, 7], and a reader will want to see them. We must be explicit about a caveat: in our pipeline they perform single-step updates over the same *offline* buffer rather than interacting with a live environment, because the whole premise of the study is that online interaction is unavailable. They are therefore reference points, not a fair online comparison, and we never claim otherwise. PPO and SAC use a Dirichlet actor (concentration $\text{softplus}(f_\phi(s)) + 1$) and act with the Dirichlet mean; A2C uses a softmax actor.

2.2.6 Classical baselines

Six parameter-free allocators provide the bar the learners must clear: **buy-and-hold** and **equal weight** (both emit $1/N$ daily—under a target-referenced turnover model these coincide); **mean-variance** (max-Sharpe Markowitz [24] on a 252-day window with a Ledoit–Wolf shrunk covariance [20], solved by 200 projected gradient steps on the simplex); **minimum-variance** (same solver, variance objective); **risk parity** (inverse 60-day volatility); and **momentum** (equal weight over the top $\lfloor N/3 \rfloor = 4$ assets by 60-day trailing return).

2.2.7 Hyperparameters

Table 2 lists every hyperparameter. The transformer shape ($K = 30$, $d_{\text{model}} = 192$, 4 layers, 6 heads) was chosen to fit the 8 GB GTX 1070 while leaving room for a GPU-resident replay buffer, and K is ablated in Section 3.7.3. The optimiser is AdamW [25] at 10^{-4} with weight decay 10^{-4} and a cosine

Table 2: Hyperparameters. All learned models share the training budget, the seed set and the batch size, so differences between them are attributable to the algorithm rather than to tuning effort.

Group	Hyperparameter	Value	Why
Environment	transaction cost μ	0.001 (10 bp)	retail-realistic; ablated over $\{0, 5, 10, 25\}$ bp
	reward floor ε	10^{-8}	keeps $\ln(\cdot)$ finite on crash days
	episode length T_{ep}	252	one trading year; also the RTG reset horizon
	episode stride	21	one month; decorrelates sampled windows
Transformer	context length K	30	90 tokens; ablated over $\{10, 20, 30, 50\}$
	d_{model} / layers / heads	192 / 4 / 6	2.30 M params, fits 8 GB with the buffer resident
	dropout	0.1	mild regularisation; train/val curves stay together
	optimiser	AdamW, cosine	[25]; DT paper’s choice
	learning rate / weight decay	10^{-4} / 10^{-4}	stable at this model size
	gradient clip	1.0	prevents rare large-loss spikes
RL agents	hidden width / depth	256 / 2	matched across TD3+BC, IQL, CQL, PPO, SAC
	discount γ	0.99	standard; DT uses $\gamma = 1$ for RTG
	Polyak τ	0.005	standard TD3 setting
	TD3+BC α / IQL expectile	2.5 / 0.7	paper defaults
	CQL α_{CQL} / # actions	1.0 / 10	CQL(H) over sampled simplex actions
Training	seeds	$\{42, 123, 456, 789, 1011\}$	5 runs per model; we report mean $\pm$ std
	epochs $\times$ steps/epoch	50×200	~ 3 passes over 816k transitions
	batch size	256	throughput plateaus at ~ 1300 samples/s
	mixed precision	disabled	Pascal fp16 is $\sim 1/64$ of fp32 throughput
Inference	target RTG quantile	0.90	of training-episode starting RTG; ablated

schedule over 50 epochs, with gradient-norm clipping at 1.0; these are the Decision Transformer paper’s settings scaled down for our model size. Batch size 256 was selected empirically: measured throughput on this GPU plateaus near 1,300 samples/s for every batch size from 128 to 1024, so larger batches buy only VRAM. Mixed precision is deliberately disabled—the Pascal GP104 chip has no tensor cores and runs fp16 at roughly 1/64 of fp32 throughput, so AMP would slow training down. Finally, `steps_per_epoch` is capped at 200: a full pass over the buffer is 3,189 steps, which would make the five-seed sweep a ~ 87 -hour job, whereas 50×200 steps still covers about three full passes and fits in an overnight window.

2.3 Performance Evaluation

Data partition. The split is strictly chronological, never random, because randomly shuffled financial data leaks the future into the past through overlapping feature windows. The **training** period is 2000-09-01 to 2016-12-31, the **validation** period is 2017-01-01 to 2019-12-31, and the **test** period is 2020-01-01 to 2025-12-30 (1,507 trading days). Feature standardisation statistics, behaviour-policy rollouts and all offline episodes come from the training period alone. Validation is used only for model selection—the checkpoint with the lowest validation action MSE is the one that gets backtested—and is never backtested for reporting. Beyond this fixed holdout we also run a **walk-forward** evaluation over six two-year blocks (2014–15, 2016–17, 2018–19, 2020–21, 2022–23, 2024–25); the first two begin before the training cutoff and are flagged in-sample, and we report them only as a sanity check.

Error function. The training objective for DT and BC is the mean squared error between the predicted allocation and the behaviour action, $\mathcal{L}(\theta) = \mathbb{E}_{\tau \sim \mathcal{D}} [\|a_t - \hat{f}_\theta(\hat{R}_{t-K+1:t}, s_{t-K+1:t}, a_{t-K+1:t-1})\|_2^2]$, evaluated at the last context position. The RL agents instead minimise their algorithm-specific temporal-difference and actor losses over the same transitions. Note that a low MSE means faithful *imitation*, not good *allocation*—this distinction turns out to matter, and Section 3.7.3 shows the two moving in opposite directions.

Financial metrics. Every strategy is scored on the same log-return series with the same code: cumulative return, CAGR, annualised volatility, **Sharpe ratio** [26] (our headline metric, computed against a 2% annual risk-free rate on a 252-day year), Sortino ratio, maximum drawdown, Calmar ratio, 95% VaR and CVaR, and mean daily L_1 turnover. We lead with Sharpe rather than return because with a simplex action space any strategy can raise its return by concentrating into whatever rallied, and Sharpe is what penalises that.

Statistical testing. Because daily returns are fat-tailed (Figure 3) and serially dependent, we do not test Sharpe differences with a Gaussian assumption. We report (i) 95% confidence intervals for the Sharpe difference against buy-and-hold from a circular block bootstrap (block length 20, 1,000 replicates) in the spirit of [27], (ii) the Ledoit–Wolf robust Sharpe-difference test [18], and (iii) the deflated Sharpe ratio [19] with $n_{\text{trials}} = 46$, the total number of strategy variants evaluated. Reporting a multiple-testing correction is essential here: a study that trains 40 checkpoints and reports the best one is guaranteed to find something.

3 Experimental Analysis

Code, configuration and all result tables are available at

<https://github.com/ShojebBhuiyan/Decision-Transformer-for-Sequential-Portfolio-Allocation-using>

The full pipeline is reproducible with `python main.py --stage all`; the ablation sweeps of Section 3.7 are produced by `scripts/run_paper_ablations.py` and the figures by `scripts/run_paper_figures.py`. A suite of 58 unit tests covers the correctness safeguards (simplex validity, hand-computed rewards, no-lookahead features, the RTG recursion, split-boundary leakage, and the RL fixes of Section 3.8).

3.1 Convergence

Figure 6 shows the convergence of the two transformer models. Both converge cleanly and identically. Averaged over seeds, DT ends at a training MSE of 3.245×10^{-3} and a best validation MSE of 3.245×10^{-3} ; BC ends at 3.263×10^{-3} and 3.257×10^{-3} . Validation loss reaches its minimum at epoch 47–50 in every one of the ten runs, so no run was stopped early by the checkpoint-selection rule and none diverged. Since the validation curve never separates from the training curve, the generalisation gap on the *imitation* task is essentially zero—which, as the backtests will show, does not translate into generalisation on the *allocation* task.

3.2 Classical baselines on the 2020–2025 holdout

Table 3 establishes the bar. Equal weight earns a Sharpe of 1.118 with a CAGR of 23.7% and a maximum drawdown of -23.3% , and neither Markowitz optimisation nor minimum-variance improves on it in any statistically meaningful way: their Sharpe differences are -0.0028 with bootstrap CIs of $[-0.066, 0.052]$ and $[-0.011, 0.007]$ and Ledoit–Wolf p -values of 0.91 and 0.97. This is the classic “ $1/N$ is hard to beat” result [1] reproduced on our universe, and it is why the

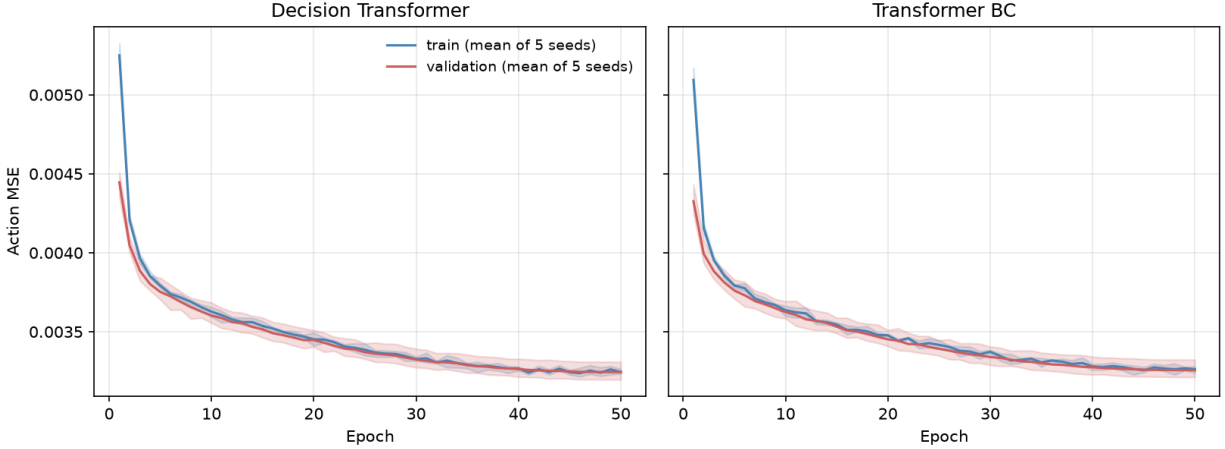


Figure 6: Training and validation action-MSE for the Decision Transformer (left) and the architecturally identical RTG-free control (right). Solid lines are the mean over the five seeds $\{42, 123, 456, 789, 1011\}$; shaded bands span the seed-wise minimum and maximum. Both models fall steeply for roughly 10 epochs and then flatten, converging to $\approx 3.2 \times 10^{-3}$ by epoch 50, and the validation curve tracks the training curve for the whole run with no upward turn—i.e. neither model overfits the offline buffer, and the 50-epoch budget is sufficient. Two observations matter for the rest of the paper. First, the two panels are nearly superimposable: adding the Return-to-Go channel lowers the best validation MSE by only 1.2×10^{-5} (a 0.4% relative improvement), the first sign that the conditioning signal carries almost no usable information. Second, the seed spread on the loss is far narrower than the seed spread we later see on backtest Sharpe (± 0.13), which means the variance in Table 4 comes from how small weight differences compound over a 1,507-day rollout, not from unstable optimisation.

study needs significance testing rather than a leaderboard. Momentum produces the highest raw growth (27.1% CAGR) but pays for it with 22.6% volatility, a -32.0% drawdown and 15.1% mean daily turnover, so its Sharpe is 0.973—worse than doing nothing, with $p = 0.0002$. Risk parity fails outright (0.6% CAGR, Sharpe -0.178): inverse-volatility weighting pours capital into the cash and bond sleeve, which is precisely the wrong place to be during the 2020–2025 equity run.

3.3 Learned allocators

Table 4 is the central result, and it is negative: **no learned allocator beats cost-aware equal weight on Sharpe**. The ordering is itself informative. PPO ties the baseline exactly, but only because its Dirichlet concentration parameters collapse so that the mean action is $1/N$ on every seed—it is not allocating, it is abstaining, and the zero standard deviation is the tell. SAC is the best genuinely active learner (1.039 ± 0.160) and still trails. The Decision Transformer reaches 0.966 ± 0.133 against its RTG-free control’s 0.976 ± 0.106 : a gap of -0.010 Sharpe, an order of magnitude smaller than the seed-to-seed spread of either model. The value-based offline methods are clearly worse: IQL 0.818, CQL 0.781, TD3+BC 0.660, all with much higher turnover (12.7–19.5% daily) and deeper drawdowns. CQL deserves a separate note because it is the one row where a learned method looks impressive on the wrong axis: its mean CAGR of 34.3% is the highest number anywhere in this paper, but it is bought with 34.6% volatility and a -49.9% maximum drawdown, i.e. CQL’s conservatism penalty pushes it into a few concentrated bets that happened to work in this particular window. A2C is unusable, with mean daily turnover above 1.0 (it rewrites the entire portfolio every day) and negative CAGR.

One caveat on the transformer rows: their turnover is bimodal across seeds, not merely noisy.

Table 3: Universe A, held-out test period 2020-01-01 to 2025-12-30 (1,507 trading days), $\mu = 10$ bp. Turnover is mean daily L_1 turnover. Buy-and-hold and equal weight are identical under a target-referenced turnover model and are reported once.

Strategy	Cum. ret.	CAGR	Vol.	Sharpe	Sortino	Max DD	Turnover
Buy-and-hold / $1/N$	2.569	0.237	0.172	1.118	1.379	−0.233	~0
Mean-variance	2.647	0.242	0.176	1.116	1.393	−0.233	0.007
Minimum-variance	2.527	0.235	0.171	1.116	1.372	−0.233	0.0002
Momentum	3.199	0.271	0.226	0.973	1.311	−0.320	0.151
Risk parity	0.034	0.006	0.081	−0.178	−0.145	−0.233	0.001

Table 4: Universe A test period, learned allocators: mean $\pm$ standard deviation across the five seeds. Every row is a backtest of a trained checkpoint through the same environment used for Table 3. The buy-and-hold reference line is repeated for convenience. [†]PPO’s Dirichlet mean collapses to $1/N$ on all five seeds, so it reproduces buy-and-hold exactly (zero variance) by degenerating, not by allocating.

Model	CAGR	Volatility	Sharpe	Max DD	Turnover
<i>Buy-and-hold (reference)</i>	0.237	0.172	1.118	−0.233	~0
PPO [†] (online ref.)	0.237 $\pm$ 0.000	0.172	1.118 $\pm$ 0.000	−0.233	~0
SAC (online ref.)	0.222 $\pm$ 0.029	0.175	1.039 $\pm$ 0.160	−0.248 $\pm$ 0.044	0.058
Transformer BC	0.219 $\pm$ 0.009	0.184	0.976 $\pm$ 0.106	−0.267 $\pm$ 0.048	0.059
Decision Transformer	0.215 $\pm$ 0.011	0.183	0.966 $\pm$ 0.133	−0.272 $\pm$ 0.062	0.061
IQL	0.196 $\pm$ 0.035	0.199	0.818 $\pm$ 0.204	−0.303 $\pm$ 0.025	0.127
CQL	0.343 $\pm$ 0.117	0.346	0.781 $\pm$ 0.214	−0.499 $\pm$ 0.098	0.195
TD3+BC	0.190 $\pm$ 0.060	0.238	0.660 $\pm$ 0.261	−0.351 $\pm$ 0.033	0.135
A2C (online ref.)	−0.014 $\pm$ 0.113	0.307	−0.127 $\pm$ 0.378	−0.539 $\pm$ 0.165	1.139

Seeds 123, 456, 789 and 1011 all produce ~4.3% mean daily turnover and Sharpe 0.95–1.09, while seed 42 produces 13.6% (DT) and 12.3% (BC) and Sharpe 0.75/0.81. The mode is a property of the *seed*, not of the architecture—the same seed puts DT and BC in the same mode—so the 0.061 and 0.059 turnover figures in Table 4 are averages over two distinct behaviours rather than descriptions of a typical run. We return to this in Section 3.7.3, where it limits what the context-length sweep can establish.

Figure 7 shows the same result as wealth paths. The learned transformers do not fail catastrophically—they track the market, which is what a behaviour-cloned policy over a mixture dominated by sensible allocators should do—but they shed a few percent of terminal wealth to turnover and to small timing errors, with the gap opening in the March 2020 crash and never closing.

3.4 What the learned allocators actually hold

Aggregate performance metrics say what happened; Table 5 says why. Four readings stand out.

PPO’s collapse is exact. Its HHI equals $1/14$ on all five seeds and its largest mean weight is 0.0714, so the “tie” with buy-and-hold in Table 4 is arithmetic identity, not a coincidence of performance.

DT and BC do not merely track equal weight—on four of five seeds they *are* equal weight plus noise. At seeds 123, 456, 789 and 1011 their HHI is 0.0715 against a uniform 0.0714, the largest mean weight is 7.5–7.9% against a uniform 7.1%, and the three largest holdings are effectively arbitrary: Walmart and silver at one seed, JP Morgan and cash at another, Nvidia and Microsoft at a third, with nothing above eight percent anywhere. Only the fifth seed—the

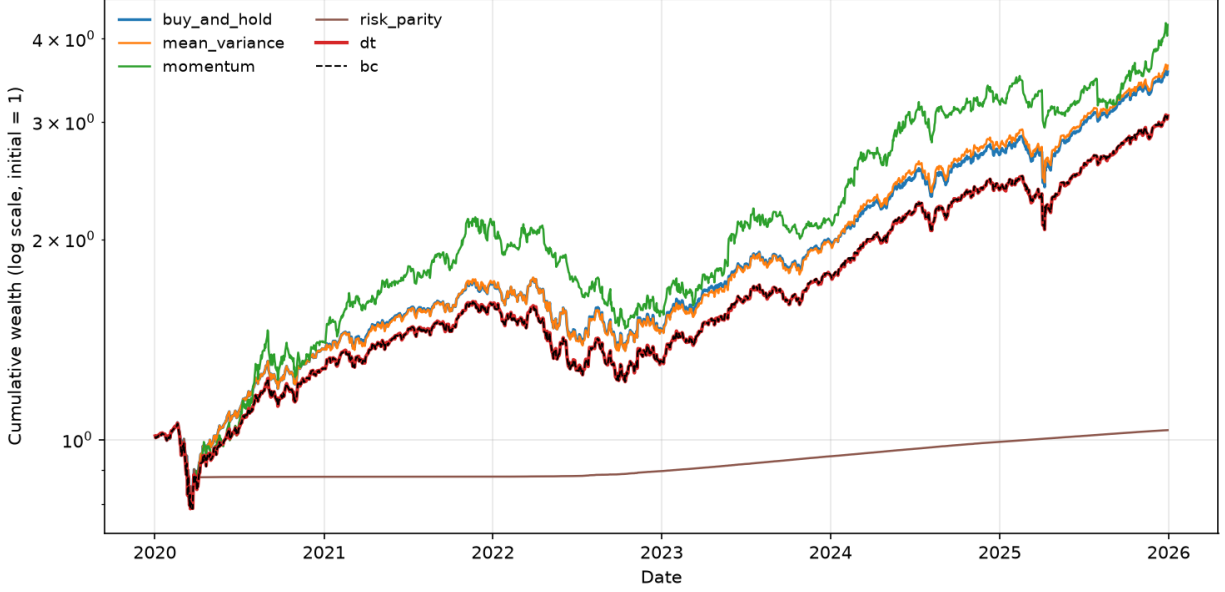


Figure 7: Cumulative wealth on the held-out window (log scale, initial capital = 1, $\mu = 10$ bp). DT and BC are shown at seed 456, the median-Sharpe run; seed 42 is the high-turnover outlier discussed in Section 3.3, and plotting it would misrepresent a typical rollout. The visual story matches Tables 3–4: buy-and-hold and mean-variance are indistinguishable; momentum leads throughout on level but with visibly larger swings, including a sharper 2022 drawdown; and DT and BC track the market closely while sitting persistently *below* it, the gap opening in the March 2020 crash and never closing. Risk parity flatlines just above its starting capital for all six years. BC (dashed black) is drawn over DT (red) because the two wealth paths are otherwise indistinguishable—the clearest single illustration that return conditioning changes almost nothing.

high-turnover run of Section 3.3—holds a real megacap-growth tilt (HHI = 0.085, Nvidia at 14%). The typical outcome of training a Decision Transformer on this buffer is therefore a policy that reproduces the baseline’s allocation and then pays 4.3% daily turnover to keep re-deriving it, which is precisely the cost drag that Section 3.7.4 isolates. DT and BC are also indistinguishable seed by seed, agreeing to the fourth decimal of HHI—a third independent confirmation of the RTG result, after the loss curves and the Sharpe comparison.

SAC scores best among the active learners because it deviates least (HHI = 0.073, largest mean weight under 10%). Its tilt is not even consistent across seeds—copper and gold at one, the energy fund and silver at another—which is another way of saying it has not learned a view.

CQL is not being conservative at all. Its HHI averages 0.232—more than three times uniform—and reaches 0.283 at seed 789, where a single position averages 42% of the book. The concentration is systematic rather than random: on every one of the five seeds the three largest holdings are Nvidia, Amazon and Apple, together 63–84% of capital. Its 34.3% CAGR and –49.9% maximum drawdown are one repeated bet on the 2020–2025 technology rally, not evidence that pessimistic offline RL extracts a better policy. Universe B makes the point unmistakable: CQL’s average book there is 80.2% Nvidia and 13.2% Bitcoin.

Table 5: Mean allocation held over the test window, summarised by the Herfindahl–Hirschman index $\text{HHI} = \sum_i \bar{w}_i^2$, which equals $1/N = 0.0714$ for a perfectly uniform book and 1 for a single-asset book. Every learned row aggregates all five seeds; the bracketed range is the seed minimum and maximum. This table explains the Sharpe ordering of Table 4 better than any training metric does.

Model	HHI (mean [min, max])	Max mean weight	Turn-over	What the book looks like
Equal weight ($1/N$)	0.0714	0.071	~ 0	uniform by construction
PPO	0.0714 [0.0714, 0.0714]	0.071	~ 0	uniform on every seed
SAC	0.0732 [0.0723, 0.0741]	0.097	0.058	near-uniform; tilt varies by seed
BC	0.0742 [0.0715, 0.0846]	0.090	0.059	$1/N$ + noise on 4 of 5 seeds
DT	0.0742 [0.0715, 0.0846]	0.090	0.061	$1/N$ + noise on 4 of 5 seeds
IQL	0.0843 [0.0816, 0.0871]	0.160	0.127	cash-heavy
TD3+BC	0.0865 [0.0820, 0.0932]	0.152	0.135	Nvidia / Apple / cash
A2C	0.1196 [0.1050, 0.1303]	0.207	1.139	cash + megacaps, rewritten daily
CQL	0.2321 [0.2019, 0.2826]	0.321	0.195	Nvidia, Amazon, Apple on <i>every</i> seed

Table 6: Significance of each learned model’s Sharpe against buy-and-hold on the test window. “CI excludes 0” counts, out of five seeds, how many circular block-bootstrap 95% intervals for the Sharpe difference lie entirely below zero; “ $p < 0.05$ ” counts the Ledoit–Wolf robust tests that reject equality. No seed of any model produced an interval lying entirely *above* zero.

Model	Mean ΔSharpe vs. $1/N$	CI excludes 0 (worse)	Ledoit–Wolf $p < 0.05$
SAC	−0.079	1/5	4/5
Transformer BC	−0.142	3/5	3/5
Decision Transformer	−0.152	3/5	3/5
IQL	−0.300	1/5	4/5
CQL	−0.338	0/5	4/5
TD3+BC	−0.458	3/5	5/5
A2C	−1.246	5/5	5/5

3.5 Statistical significance

Table 6 confirms that the gaps in Table 4 are not an artefact of a lucky window. Every model has a negative mean Sharpe difference, no model ever produces a confidence interval strictly above zero, and for the two worst models the rejection is unanimous across seeds. CQL is the interesting case: its bootstrap intervals are so wide (e.g. $[-1.26, 0.041]$ at seed 123) that none excludes zero even though four of five Ledoit–Wolf tests reject—a direct measurement of how much of CQL’s apparent performance is noise. The deflated Sharpe ratio is uninformative at this scale and we report it for completeness only: with $n_{\text{trials}} = 46$, the DSR is numerically zero for every strategy *including buy-and-hold*. Counting each seed as an independent trial is too conservative; an algorithm-level $n_{\text{trials}} = 14$ would be the more defensible choice, and we flag this as a methodological limitation rather than quietly dropping the statistic.

3.6 Walk-forward evaluation

Table 7 checks whether the headline result is an accident of the 2020–2025 window. It is not. Across the four out-of-sample folds, DT’s Sharpe is within 0.10 of BC’s in every single one, and both trail $1/N$ in every single one. The folds also expose momentum’s regime dependence in a way the aggregate table hides: momentum wins 2018–19 and 2020–21 by chasing the megacap-growth

Table 7: Walk-forward Sharpe over six two-year blocks; learned columns are means across five seeds. The first two folds begin before the 2016-12-31 training cutoff and are marked in-sample (shown for context only); the last four are genuinely out of sample. The pattern is stable across every out-of-sample fold: learned allocators track $1/N$ but never lead it.

Fold	Status	$1/N$	Mean-var.	Mom.	DT	BC	SAC	CQL
2014–15	in-sample	0.413	0.469	0.627	0.379	0.378	0.382	1.375
2016–17	in-sample	2.403	2.384	1.994	2.337	2.352	2.383	1.031
2018–19	out-of-sample	0.802	0.800	0.861	0.790	0.777	0.785	0.186
2020–21	out-of-sample	1.243	1.234	1.318	1.068	1.056	1.092	0.953
2022–23	out-of-sample	0.329	0.323	0.069	0.254	0.277	0.328	−0.022
2024–25	out-of-sample	1.941	1.906	1.384	1.838	1.849	1.870	1.442

trend, then collapses to 0.069 in the 2022–23 reversal while equal weight holds 0.329. The two in-sample folds are a useful negative control: CQL posts its best Sharpe anywhere (1.375 in 2014–15) on data that overlaps its training window and then falls apart out of sample, which is what overfitting looks like when you can see both sides of the cutoff.

3.7 Ablation studies

We ablate four factors—the RTG conditioning channel, the target RTG value, the transformer context length, and the transaction cost—plus a complete replication on a different asset universe.

3.7.1 Ablation 1: does Return-to-Go conditioning do anything?

Table 8: RTG conditioning on/off. The two models are the same architecture, parameter count, optimiser, batches and seeds; only the Return-to-Go input differs.

	Params	Best val. MSE	Test Sharpe	Test CAGR
DT (RTG on)	2,298,638	3.245×10^{-3}	0.966 ± 0.133	0.215 ± 0.011
BC (RTG off)	2,298,638	3.257×10^{-3}	0.976 ± 0.106	0.219 ± 0.009
Difference (DT – BC)	0	-1.2×10^{-5}	−0.010	−0.004

This is the ablation the paper was designed around. Table 8 shows that removing the Return-to-Go channel costs almost nothing. Conditioning does buy a marginally better fit to the behaviour data—the best validation MSE is 1.2×10^{-5} (0.4%) lower with RTG on—but that fit does not become allocation skill: DT’s test Sharpe is *lower* by 0.010, against seed standard deviations of 0.11–0.13. In other words the extra channel is used just enough to memorise the buffer slightly better and not at all to allocate better, which is the signature of an input the policy head has learned to ignore.

3.7.2 Ablation 2: sweeping the target Return-to-Go

Table 9 strengthens the previous result. Moving the target from the 10th to the 90th percentile of training returns—a swing of 0.51 in log-return units, i.e. asking for a wildly different outcome—changes the realised cumulative log return by 0.013 and the Sharpe by 0.040, and the slope is *negative*: asking for more return gets slightly less. Figure 8 plots this against the identity line

Table 9: Target-RTG sweep. The conditioning target is set to a quantile of the training episodes’ starting Return-to-Go and reset every 252 days; each row averages five seeds. If return conditioning worked, realised return would increase down the table.

Quantile	Target $\hat{R}_0$	Realised cum. log return	Realised CAGR	Sharpe
0.10	−0.262	1.179	0.218	1.006
0.25	−0.123	1.177	0.218	1.004
0.50	+0.006	1.173	0.217	1.001
0.75	+0.130	1.166	0.215	0.967
0.90	+0.248	1.166	0.215	0.966
<i>range across a 0.51 swing in target</i>		0.013	0.003	0.040

the model would follow if conditioning were perfectly calibrated. Three mechanisms plausibly contribute. First, the RTG scalar is standardised with $\sigma_{\text{RTG}} = 0.139$ estimated over 252-day episodes, which compresses the quantile range into a narrow band of input values. Second, resetting the budget every 252 days means no single target ever governs more than one year of a six-year rollout. Third and most fundamentally, the training loss is MSE against the behaviour action: nothing in the objective rewards the model for *changing* its action when $\hat{R}$ changes, so the cheapest way to minimise the loss is to ignore the RTG token. In an environment with an exogenous price process, unlike Atari or locomotion, there may simply be no action-conditional path to a higher return for the model to discover.

3.7.3 Ablation 3: context length K

Table 10: Context-length ablation. The Decision Transformer is retrained from scratch at each K with every other hyperparameter and the seed (42) held fixed; $K = 30$ reuses the main `dt_seed42` checkpoint, whose configuration is identical. Losses are the final training and best validation action MSE; the remaining columns are the 2020–2025 backtest.

K	Tokens $3K$	Train MSE ($\times 10^{-3}$)	Val. MSE ($\times 10^{-3}$)	Sharpe	CAGR	Max DD	Turnover
10	30	3.202	3.298	0.949	0.218	−0.288	0.043
20	60	3.224	3.303	0.941	0.216	−0.289	0.042
30	90	3.242	3.309	0.747	0.203	−0.374	0.136
50	150	3.225	3.301	0.667	0.178	−0.378	0.146

Table 10 and Figure 9 vary the one architectural hyperparameter that is specific to the sequence-modelling framing. Two readings are warranted, and they differ in how confident we can be.

The **unambiguous** result is that the imitation loss is flat in K . Going from 30 tokens to 150 changes the best validation MSE by 0.011×10^{-3} , a relative change of 0.3%, and the ordering is not even monotone. Whatever the transformer is using to predict the next allocation, it is essentially contained in the most recent ten days; the additional forty days of context available at $K = 50$ are not informative. That is consistent with the state vector already carrying a 20-day lookback internally (Section 2.1), so a longer token context largely duplicates information the state encoding provides.

The **suggestive but confounded** result is the Sharpe column, which falls monotonically from 0.949 at $K = 10$ to 0.667 at $K = 50$, tracking a jump in mean daily turnover from 4.3% to 14.6%. Taken at face value this would say that longer contexts make the policy churn. We do not claim

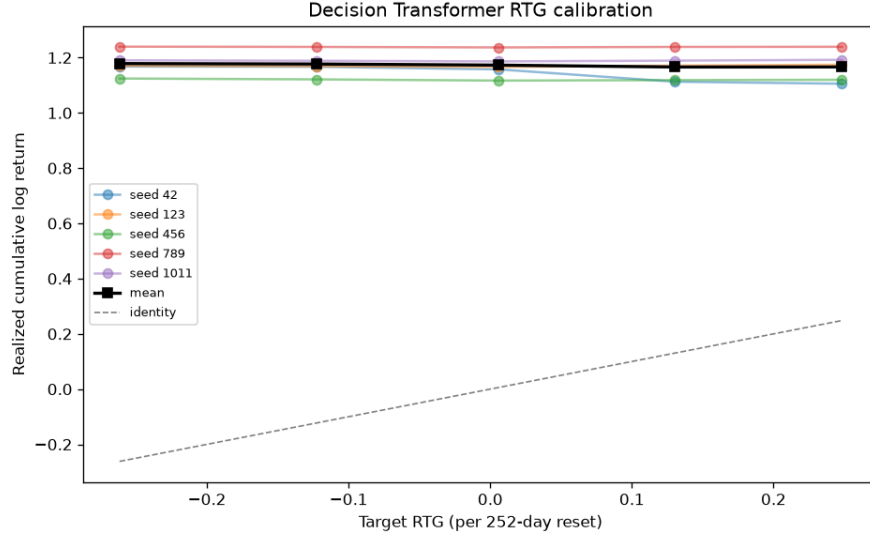


Figure 8: Return-to-Go calibration for the Decision Transformer. Each coloured line is one seed; the heavy black line is the seed mean; the dashed grey line is the identity a perfectly calibrated model would follow (realised return = requested return). The realised curves are flat: over the entire range of targets the model produces the same allocation policy, and the mean line drifts slightly *downwards* rather than upwards. The large vertical offset between the flat realised lines and the identity line is a separate, expected artefact—the target is a one-year budget while the realised value accumulates over six years—but the absence of any positive slope is the finding.

that, because the sweep uses one seed per K and the five seeds trained at $K = 30$ are themselves bimodal in exactly this way: seed 42 turns over 13.6% per day and scores 0.747, while seeds 123, 456, 789 and 1011 all turn over 4.2–4.4% and score 0.95–1.09. The split is caused by the seed rather than by the model: at seed 42 the RTG-free control lands in the same high-turnover mode (12.3%, Sharpe 0.81), and at every other seed both models land in the low one. The high-turnover mode is therefore a seed-dependent failure mode that exists at fixed K , and with one run per setting we cannot separate it from a genuine effect of K . The honest summary is that the $K = 10$ and $K = 20$ runs landed in the well-behaved mode and the $K = 30$ and $K = 50$ runs did not. It is also the one place in this study where a larger compute budget would clearly have changed what we could conclude: five seeds per K would settle it, and at roughly 45 minutes per $K = 50$ run on a GTX 1070 that was out of reach here.

Two things survive both readings. No context length produced a Sharpe above the $K = 30$ five-seed mean of 0.966, and none came close to the 1.118 baseline, so the default $K = 30$ is not leaving performance on the table. And the bimodality itself is worth recording: DT training on this buffer converges to two distinct policies with indistinguishable imitation loss (3.3×10^{-3} in both modes) but a threefold difference in turnover and a 0.3 difference in Sharpe. A validation loss computed on held-out *transitions* cannot tell these apart, which is a concrete argument for selecting offline-RL checkpoints on a backtest statistic rather than on the training objective.

3.7.4 Ablation 4: transaction cost sensitivity

Table 11 and Figure 10 separate two explanations that are easy to conflate. Costs clearly matter, and they matter in direct proportion to turnover: the Sharpe lost per 25 bp is -0.421 for momentum (15.1% daily turnover), -0.202 and -0.194 for DT and BC (6.1% and 5.9%), -0.024 for mean-

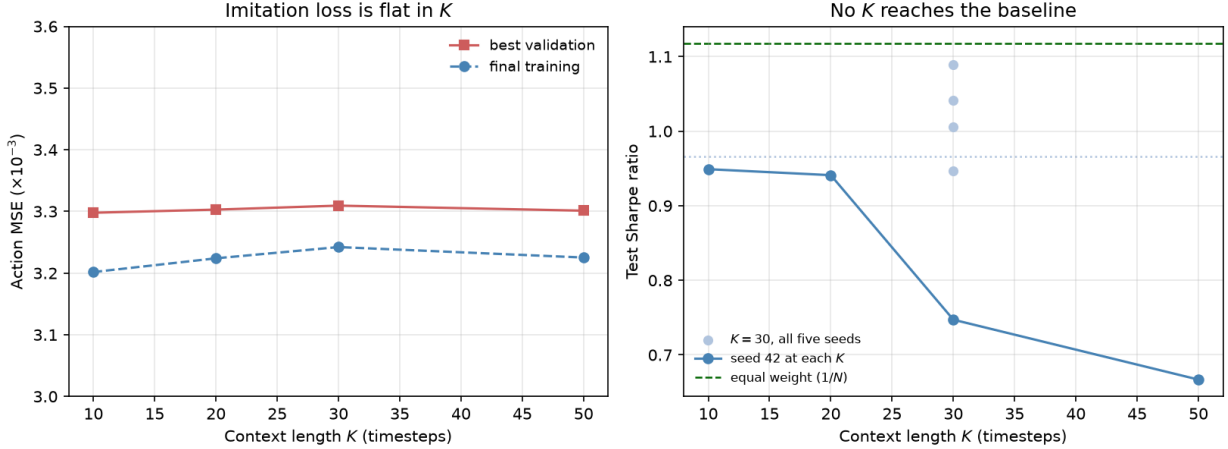


Figure 9: Context-length ablation. **Left:** final training and best validation action MSE against K . Over a five-fold range of context the loss moves by less than 1% ($3.298\text{--}3.309 \times 10^{-3}$ on validation) and does not decrease with K —extra history buys the model no additional ability to reproduce the behaviour policies. **Right:** test Sharpe at each K (seed 42, solid line) against the five individual seeds trained at $K = 30$ (light dots, dotted line at their mean 0.966) and the equal-weight baseline (dashed green). The seed-42 curve falls steeply with K , but the $K = 30$ seed cloud spans 0.75–1.09, which is wider than the $K=10$ -to- $K=20$ gap—so the drop must be read with the caveat in the text. What the panel does show unambiguously is that no context length, and no seed at any context length, reaches the baseline.

variance (0.7%) and exactly zero for equal weight. Momentum is the clearest casualty. At zero cost it is the best strategy anywhere in this study—Sharpe 1.141 against equal weight’s 1.118—and that edge is gone by roughly 1.4 bp; only its raw CAGR advantage survives, and only to about 17 bp.

But costs do *not* explain the learned models’ failure; they explain about half of it. At $\mu = 0$, where trading is free, DT averages 1.047 ± 0.096 and BC 1.054 ± 0.079 against equal weight’s 1.118. Decomposing DT’s total deficit of 0.152 Sharpe at the default 10 bp: 0.072 of it is holding a worse portfolio than $1/N$ even when rebalancing is free, and the remaining 0.081 is the cost of the churn required to hold it. That is a useful bound on the obvious fix—a turnover-penalised or trade-throttled DT could recover a little over half the gap and would still sit behind a policy that does nothing at all.

3.7.5 Ablation 5: a different asset universe

Table 12 repeats the entire study on a universe that adds two assets with very different dynamics—Bitcoin, which is roughly four times as volatile as the equity sleeve, and aluminium—and which, because Bitcoin only starts in late 2014, leaves a much shorter training history. Every qualitative conclusion survives: equal weight is again at the top, no learned model beats it, CQL again buys a spectacular CAGR (65.2%, driven by an 80.2% average weight in Nvidia plus 13.2% in Bitcoin) with a catastrophic -64.8% drawdown and a mediocre Sharpe, PPO again collapses to $1/N$, and A2C is again unusable. Two differences are worth noting. First, DT now edges *ahead* of BC (1.091 vs. 1.083), reversing the Universe A ordering—but the gap is 0.008 against seed standard deviations of 0.12, so the honest reading is that the two are indistinguishable in both universes rather than that conditioning helps here. Second, the high-turnover failure mode does not occur at all in Universe B: all five DT seeds turn over 4.6% per day, with no outlier. The 6.1% Universe A mean in Table 4 is entirely the seed-42 outlier, so the apparent improvement is the *absence* of a bad seed rather than

Table 11: Transaction-cost ablation on the test window. Each column re-runs the full backtest with a different proportional cost μ ; DT and BC rows average all five seeds. The Δ Sharpe column is the Sharpe lost per 25 bp of additional cost, which tracks the strategy’s mean daily turnover almost exactly.

Strategy	Sharpe at cost μ				Δ Sharpe / 25 bp	Turnover
	0 bp	5 bp	10 bp	25 bp		
Buy-and-hold / $1/N$	1.118	1.118	1.118	1.118	0.000	~ 0
Minimum-variance	1.116	1.116	1.116	1.115	−0.001	0.0002
Mean-variance	1.125	1.121	1.116	1.101	−0.024	0.007
Momentum	1.141	1.057	0.973	0.720	−0.421	0.151
Transformer BC	1.054	1.015	0.976	0.860	−0.194	0.059
Decision Transformer	1.047	1.006	0.966	0.845	−0.202	0.061

Table 12: Universe B robustness check: Universe A plus Bitcoin and aluminium ($N = 16$), which forces the data to start on 2014-09-17 and shortens training to two years before the validation split. The entire pipeline—prepare, trajectories, a fresh 5×8 training sweep, and evaluation—was re-run into scoped artefact directories. Test window and cost are unchanged.

Strategy	CAGR	Sharpe	Max DD	Turnover
Mean-variance	0.267	1.202	−0.265	0.009
Buy-and-hold / $1/N$	0.257	1.202	−0.254	~ 0
PPO [†]	0.257 ± 0.000	1.202 ± 0.000	−0.254	~ 0
Minimum-variance	0.254	1.200	−0.251	0.0002
Momentum	0.308	1.109	−0.374	0.151
Decision Transformer	0.232 ± 0.016	1.091 ± 0.117	-0.262 ± 0.015	0.046
Transformer BC	0.230 ± 0.019	1.083 ± 0.127	-0.260 ± 0.014	0.047
SAC	0.222 ± 0.014	1.040 ± 0.195	-0.293 ± 0.060	0.064
CQL	0.652 ± 0.076	0.950 ± 0.079	-0.648 ± 0.022	0.068
IQL	0.231 ± 0.060	0.838 ± 0.339	-0.393 ± 0.103	0.121
TD3+BC	0.249 ± 0.042	0.642 ± 0.128	-0.460 ± 0.076	0.143
A2C	0.189 ± 0.149	0.340 ± 0.249	-0.661 ± 0.125	1.035

a genuinely more conservative clone—a shorter, more homogeneous training period seems to make the bad mode unreachable. The Universe B RTG sweep is likewise flat, moving Sharpe from 1.084 at the 10th percentile to 1.091 at the 90th—a +0.007 slope, nominally in the right direction and numerically irrelevant.

3.8 Correctness issues that had to be fixed first

Several defects in an early version of the RL baselines made their losses meaningless, and we record them because they are easy to reproduce and hard to notice. The Bellman target used $s' = s$, so the critics solved $Q(s) = r + \gamma Q(s)$ and diverged; TD3+BC’s target critic was deep-copied once and never updated; IQL’s value function was fitted against the *live* critic instead of a frozen target and chased itself to a loss of 10^4 ; CQL’s conservative penalty took a log-sum-exp over the *batch* rather than over sampled actions, which makes the penalty approximately $2 \log(\text{batch size})$ and independent of the policy; and RL checkpoints stored the algorithm’s name string rather than its weights, so “loading” a checkpoint silently produced a randomly initialised agent. Separately, the transformer’s right-aligned padding mask could mask every key in a query row, producing a softmax over an empty set and NaN gradients. All are now fixed—real next states from the episode index

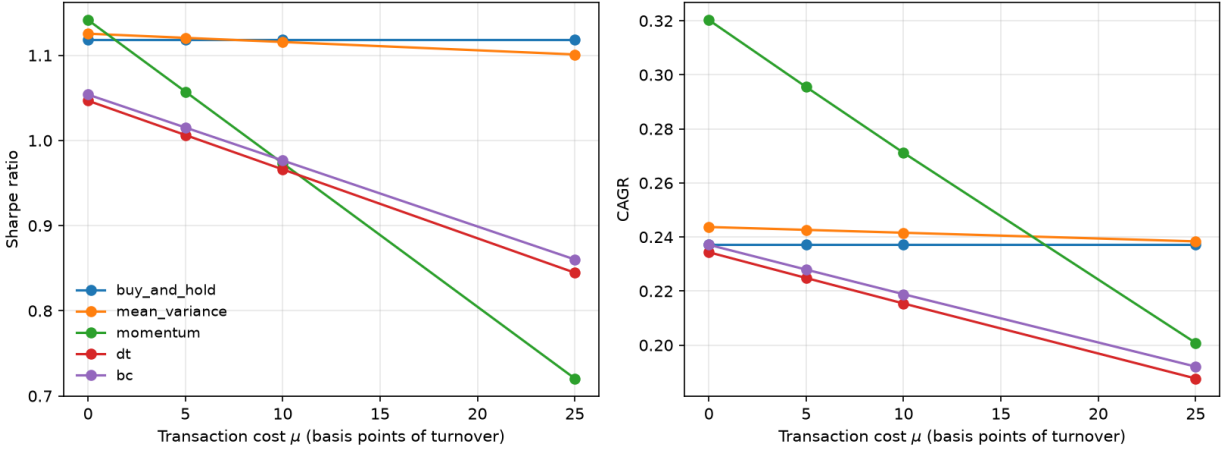


Figure 10: Sharpe (left) and CAGR (right) against the proportional transaction cost; DT and BC are means over five seeds. Slopes are proportional to turnover, so the two flat lines are the near-zero-turnover strategies and the steepest is momentum (15.1% daily turnover), with DT (6.1%) and BC (5.9%) intermediate. Two crossings matter. In the left panel momentum starts *above* equal weight at zero cost and falls below it at roughly 1.4 bp, so its entire risk-adjusted edge is a frictionless artefact; in the right panel its raw CAGR advantage survives until about 17 bp. Most importantly, the DT and BC lines never reach the buy-and-hold line even at the left edge of the plot, where trading is free—about half of their deficit is not a transaction-cost problem at all.

with terminal transitions cutting the bootstrap, Polyak targets at $\tau = 0.005$, a frozen target for IQL’s value regression, CQL(H) over ten sampled actions, full module state dicts in checkpoints, and a guarantee that the attention diagonal is always attendable—and each is covered by a regression test.

4 Conclusion

We asked whether return-conditioned sequence modelling improves offline portfolio allocation, and built the controls needed to answer it: a Decision Transformer and an architecturally identical RTG-free control, six value-based and policy-gradient RL baselines, six classical allocators, five seeds each, a 2,580-dimensional state engineered from a public 30-year daily-close dataset, a 3,600-episode offline buffer synthesised from 18 behaviour policies, a cost-aware log-growth reward, a six-year held-out window, walk-forward folds, block-bootstrap and Ledoit–Wolf significance tests, and five ablations. The answer on this task is no. No learned allocator beat cost-aware equal weight (Sharpe 1.118); the Decision Transformer reached 0.966 ± 0.133 while its RTG-free twin reached 0.976 ± 0.106 ; sweeping the conditioning target across the full range of training returns moved realised Sharpe by 0.040 in the wrong direction; a five-fold change in context length moved the imitation loss by 0.3% and closed none of the gap; removing transaction costs entirely closed only about half of it; and a second universe containing Bitcoin reproduced every one of these findings. The most useful thing we can report is the diagnosis rather than the verdict. An MSE-to-behaviour objective gives the model no incentive to respond to the Return-to-Go token, and in an environment whose price process is exogenous there may be no action-conditional route to the requested return in the first place. Looking at the weights rather than the metrics sharpens that further: on four of five seeds the trained Decision Transformer’s average allocation is equal weight to within noise, so it has not so much failed to find a good policy as rediscovered the baseline and then paid transaction

costs to maintain it.

Limitations. The dataset is close-only, so no volume or microstructure features are possible, and it is a single (if long) historical path. The environment measures turnover against the previous *target* weights rather than against mark-to-market drifted holdings, which makes buy-and-hold and daily-rebalanced equal weight numerically identical and mildly flatters the baseline; a drifted buy-and-hold was not simulated. The compute budget (one GTX 1070) capped training at roughly three passes over the buffer rather than full convergence on a large sweep, and confined hyperparameter exploration to the context-length and cost dimensions. The context-length sweep in particular runs one seed per K , which—given the bimodal turnover behaviour documented in Section 3.7.3—is not enough to attribute its Sharpe trend to K rather than to seed variation. PPO, SAC and A2C perform single-step updates over the offline buffer rather than interacting with a live environment, so they are reference points rather than a fair online comparison. The deflated Sharpe ratio was computed with a per-seed rather than per-algorithm trial count, which makes it vacuously zero for every strategy including the baseline. And the universes, while diversified, contain 14–16 assets; results on a several-hundred-asset cross-section may differ.

Future work. The diagnosis points directly at the remedies. An objective that explicitly penalises insensitivity to the Return-to-Go token—for example, an auxiliary loss that requires the predicted allocation to change when $\hat{R}$ is perturbed, or advantage-weighted rather than plain imitation—would test whether conditioning can be made to work rather than assuming it. Feeding realised return directly into the reward through a Sharpe-like differentiable utility [2, 16] instead of cloning behaviour actions is the natural alternative. Turnover-penalised or trade-throttled variants would recover the roughly 0.08 Sharpe the transformers lose to costs, and selecting checkpoints on a validation *backtest* statistic rather than on action MSE would avoid the high-turnover mode that held-out imitation loss cannot see. Beyond the model, the most valuable extensions are a mark-to-market turnover model, a larger asset cross-section, richer state features if an OHLCV dataset is substituted, an algorithm-level deflated Sharpe, and—given how strongly the 2022–23 fold separated momentum from equal weight—an explicit regime-conditional evaluation.

References

- [1] Victor DeMiguel, Lorenzo Garlappi, and Raman Uppal. Optimal versus naive diversification: How inefficient is the 1/n portfolio strategy? *The Review of Financial Studies*, 22(5):1915–1953, 2009.
- [2] John Moody and Matthew Saffell. Learning to trade via direct reinforcement. *IEEE Transactions on Neural Networks*, 12(4):875–889, 2001.
- [3] Yue Deng, Feng Bao, Youyong Kong, Zhiquan Ren, and Qionghai Dai. Deep direct reinforcement learning for financial signal representation and trading. *IEEE Transactions on Neural Networks and Learning Systems*, 28(3):653–664, 2016.
- [4] Zhengyao Jiang, Dixing Xu, and Jinjun Liang. A deep reinforcement learning framework for the financial portfolio management problem. *arXiv preprint arXiv:1706.10059*, 2017.
- [5] Zihao Zhang, Stefan Zohren, and Stephen Roberts. Deep reinforcement learning for trading. *The Journal of Financial Data Science*, 2(2):25–40, 2020.

- [6] Hongyang Yang, Xiao-Yang Liu, Shan Zhong, and Anwar Walid. Deep reinforcement learning for automated stock trading: An ensemble strategy. In *ACM International Conference on AI in Finance (ICAIF)*, 2020.
- [7] Xiao-Yang Liu, Hongyang Yang, Qian Chen, Runjia Zhang, Liuqing Yang, Bowen Xiao, and Christina Dan Wang. Finrl: A deep reinforcement learning library for automated stock trading in quantitative finance. In *ACM International Conference on AI in Finance (ICAIF)*, 2021.
- [8] Sergey Levine, Aviral Kumar, George Tucker, and Justin Fu. Offline reinforcement learning: Tutorial, review, and perspectives on open problems. *arXiv preprint arXiv:2005.01643*, 2020.
- [9] Aviral Kumar, Aurick Zhou, George Tucker, and Sergey Levine. Conservative q-learning for offline reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 1179–1191, 2020.
- [10] Ilya Kostrikov, Ashvin Nair, and Sergey Levine. Offline reinforcement learning with implicit q-learning. In *International Conference on Learning Representations (ICLR)*, 2022.
- [11] Scott Fujimoto and Shixiang Shane Gu. A minimalist approach to offline reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 34, pages 20132–20145, 2021.
- [12] Scott Fujimoto, Herke van Hoof, and David Meger. Addressing function approximation error in actor-critic methods. In *International Conference on Machine Learning (ICML)*, pages 1587–1596, 2018.
- [13] Lili Chen, Kevin Lu, Aravind Rajeswaran, Kimin Lee, Aditya Grover, Michael Laskin, Pieter Abbeel, Aravind Srinivas, and Igor Mordatch. Decision transformer: Reinforcement learning via sequence modeling. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 34, pages 15084–15097, 2021.
- [14] Michael Janner, Qiyang Li, and Sergey Levine. Offline reinforcement learning as one big sequence modeling problem. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 34, pages 1273–1286, 2021.
- [15] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017.
- [16] Damian Kisiel and Denise Gorse. Portfolio transformer for attention-based asset allocation. *arXiv preprint arXiv:2206.03246*, 2022.
- [17] Asim Islam. 30 years of stock market data (1995–2025). Kaggle, 2025. Daily closing prices for equities, funds, metals, energy, currencies, crypto and Treasury yields.
- [18] Olivier Ledoit and Michael Wolf. Robust performance hypothesis testing with the sharpe ratio. *Journal of Empirical Finance*, 15(5):850–859, 2008.
- [19] David H. Bailey and Marcos Lopez de Prado. The deflated sharpe ratio: Correcting for selection bias, backtest overfitting, and non-normality. *The Journal of Portfolio Management*, 40(5):94–107, 2014.

- [20] Olivier Ledoit and Michael Wolf. A well-conditioned estimator for large-dimensional covariance matrices. *Journal of Multivariate Analysis*, 88(2):365–411, 2004.
- [21] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- [22] Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *International Conference on Machine Learning (ICML)*, pages 1861–1870, 2018.
- [23] Volodymyr Mnih, Adria Puigdomenech Badia, Mehdi Mirza, Alex Graves, Timothy Lillicrap, Tim Harley, David Silver, and Koray Kavukcuoglu. Asynchronous methods for deep reinforcement learning. In *International Conference on Machine Learning (ICML)*, pages 1928–1937, 2016.
- [24] Harry Markowitz. Portfolio selection. *The Journal of Finance*, 7(1):77–91, 1952.
- [25] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019.
- [26] William F. Sharpe. The sharpe ratio. *The Journal of Portfolio Management*, 21(1):49–58, 1994.
- [27] Dimitris N. Politis and Joseph P. Romano. The stationary bootstrap. *Journal of the American Statistical Association*, 89(428):1303–1313, 1994.